TradeMirror

RAG: What It Is, What It Is Not, and How It Could Fit Later

What Is RAG?

RAG stands for Retrieval-Augmented Generation. It is a technique used in AI systems that combine a large language model (LLM) with a document retrieval layer.

The standard pattern:

1. A question or prompt is received
2. A retrieval system searches a document store for relevant context
3. That context is injected into the LLM prompt
4. The LLM generates a response informed by the retrieved documents

RAG is used to make LLMs more accurate and grounded when answering questions about specific knowledge bases — legal documents, product manuals, research papers, or proprietary internal data.

Common examples in practice:

- A customer support bot that retrieves relevant help articles before answering
- A research assistant that retrieves academic papers before summarizing findings
- A compliance system that retrieves regulatory documents before answering legal questions

---

What RAG Is Not

RAG is not a trading execution pattern.

RAG does not:

- React to price ticks
- Evaluate numeric indicator conditions
- Execute orders
- Operate in milliseconds

RAG involves LLM inference, which typically adds 500ms to several seconds of latency per query. This makes it fundamentally incompatible with the execution hot path of a rules-based trading bot.

The TradeMirror bot execution path is: `Tick → Indicators → Rule check → Execute`

This is deterministic, fast, and fully auditable. Every trade can be explained by the exact indicator values and rule conditions that fired. There is no AI inference on this path and there should never be.

How They Are Different

Dimension	TradeMirror Bot (Rules Engine)	RAG System
Purpose	Execute trades when conditions are met	Answer questions using retrieved context
Speed	Sub-millisecond condition evaluation	500ms to several seconds per query
Input	Structured numeric indicator values	Natural language questions or prompts
Output	Buy/Sell order + Telegram notification	Natural language response
Determinism	Fully deterministic	Non-deterministic — LLM responses vary
Auditability	Every trade fully explainable	Reasoning is partially opaque
Position	Core execution hot path	Advisory or research layer only
Latency tolerance	Zero — acts on current candle	High — seconds of latency acceptable
When it runs	On every price tick	On demand, when a question is asked

Where RAG Could Fit in TradeMirror Later

RAG is not relevant to the current build phase. However, there are legitimate future use cases where a RAG system could add value in TradeMirror. All of them are advisory. None of them are on the execution path.

Use Case 1 — Strategy Research Assistant

A RAG system built on top of a document store containing trading books, strategy notes, backtesting reports, and market research. A trader could ask: "What does my confluence strategy say about entering in a ranging regime?" The system retrieves relevant documents and generates a contextual answer. This does not affect trade execution.

Use Case 2 — Trade Journal Intelligence

TradeMirror logs every trade with indicator values and outcomes. A RAG system could retrieve past trades matching similar setups and surface patterns. Example query: "Show me past trades where MACD crossed bullish with RSI below 30 on H1 — what was the average outcome?" This is a retrospective analytical tool.

Use Case 3 — BrainBot Market Context Layer

The BrainBot system (described in Project.md) is intended to assist with directional bias, trade context, and entry evaluation. A RAG component could allow BrainBot to retrieve current regime

analysis, recent news context, or historical pattern data before generating a market narrative. This would run on a slow advisory loop — updating a bias parameter every few minutes — never on the tick-level execution path.

Use Case 4 — Regime and Narrative Interpretation

Structured indicator outputs (ADX, ATR, Bollinger width, market structure state) could be passed to an LLM with retrieved context about current conditions. The LLM generates a plain-language narrative: "EUR/USD is currently in a trending regime with expanding volatility. RSI is approaching oversold on H1." This is observability tooling, not execution logic.

The Key Constraint

If RAG is ever added to TradeMirror, it must operate on a separate advisory loop with its own latency budget. It must never sit between a condition trigger and an order submission.

The execution path is and must remain: `Tick → Indicators → Rule check → Execute`

RAG output can inform parameters that feed into rule checks — for example, whether trading is enabled at all given current regime — but the final execution decision remains deterministic and rules-based.

Summary

RAG is a powerful tool for knowledge retrieval and language generation. It is not relevant to the current TradeMirror build phase. When it does appear in TradeMirror, it will serve as an intelligence and observability layer — helping the trader understand the market, review past performance, and interpret signals — while the execution engine remains fast, deterministic, and fully auditable.

Layer	Technology	Latency	Purpose
Execution	Rust rules engine	< 1ms	Place orders on condition trigger
Analytics	Rust + Tauri events	< 5ms	Live indicator display
Alerts	Rust + Telegram HTTP	~ 1 second	Notification on condition fire
Advisory (future)	RAG + LLM	0.5 - 5 seconds	Market narrative, research, journal